

Diskrétní struktury

Část 3 Teorie grafů (4)



Vysoká škola
polytechnická
Jihlava

Reprezentace grafů

Motivace

Velké množství problémů v informatice se formuluje pomocí grafových pojmů → Je třeba mít reprezentaci grafu vhodnou pro počítačové zpracování.

Struktura grafu je zcela vyjádřena v jeho incidenci, tedy v určení krajních uzlů hran.

Reprezentace grafů

- maticová - maticový popis grafů má spíše teoretický význam, poskytuje vazbu mezi teorií grafů a lineární algebrou
- spojová - standardní způsob vyjádření grafu při realizaci grafových algoritmů, menší asymptotická paměťová složitost, lepší časová složitost algoritmů ve srovnání s maticovou reprezentací

Matice neorientovaných grafů

Definice: Necht' $G = \langle H, U, \varrho \rangle$ je neorientovaný graf s množinou hran $H = \{h_1, h_2, \dots, h_m\}$ a množinou uzlů $U = \{u_1, u_2, \dots, u_n\}$.

Matice incidence

Maticí incidence grafu G nazýváme matici $\mathbf{A} = [a_{ik}]$ rozměrů $n \times m$, jejíž prvky jsou dány vztahem:

$$a_{ik} = \begin{cases} 1 & \text{pokud hrana } h_k \text{ inciduje s uzlem } u_i \\ 0 & \text{v opačném případě} \end{cases}$$

- nelze reprezentovat smyčky

Matice sousednosti

Maticí sousednosti grafu G nazýváme čtvercovou matici $\mathbf{V} = [v_{ik}]$ řádu n , jejíž prvky jsou celá čísla daná vztahem:

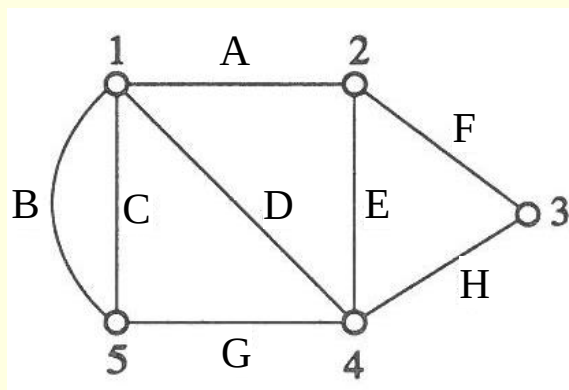
$$v_{ik} = |\varrho^{-1}([u_i, u_k])|$$

- prvek v_{ik} tedy odpovídá počtu hran incidujících s dvojicí uzlů $[u_i, u_k]$.
- matice $\mathbf{V}$ je symetrická
- pokud je graf G prostý, tak matice $\mathbf{V}$ obsahuje pouze prvky 0 a 1.

Matice neorientovaných grafů

matice incidence

matice sousednosti


$$A = \begin{array}{c|cccccccc} & A & B & C & D & E & F & G & H \\ \hline 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 4 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 5 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \end{array}$$
$$V = \begin{array}{c|ccccc} & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 & 1 & 0 \\ 3 & 0 & 1 & 0 & 1 & 0 \\ 4 & 1 & 1 & 1 & 0 & 1 \\ 5 & 2 & 0 & 0 & 1 & 0 \end{array}$$

Věta: Dva neorientované grafy jsou izomorfní právě tehdy, jsou-li jejich matice incidence stejné až na případnou permutaci (tj. "prohození") řádků a/nebo sloupců.

Matice neorientovaných grafů

Uvažujme nyní reflexivně-tranzitivní uzávěr Γ^* relace sousednosti Γ grafu G . Maticovou reprezentaci $R = [r_{ik}]$ této relace, která je ekvivalencí na množině uzlů grafu G , nazýváme **maticí dosažitelnosti** neorientovaného grafu G a její prvky mají následující význam:

$$r_{ik} = \begin{cases} 1 & \text{pokud existuje sled mezi uzly } u_i, u_k, \\ 0 & \text{v opačném případě.} \end{cases}$$

Matice orientovaných grafů

Situace je podobná jako v případě neorientovaných grafů, ale musíme uvažovat orientaci.

Definice 4.8: Nechť $G = \langle H, U, \sigma \rangle$ je orientovaný graf s množinou hran $H = \{h_1, h_2, \dots, h_m\}$ a množinou uzlů $U = \{u_1, u_2, \dots, u_n\}$. **Maticí incidence** grafu G pak nazýváme celočíselnou matici $A = [a_{ik}]$ typu (m, n) , jejíž prvky jsou dány vztahem

$$a_{ik} = \begin{cases} 1, & \text{pokud } \sigma(h_k) = (u_i, u_j) \text{ pro jisté } u_j \in U, \\ -1, & \text{pokud } \sigma(h_k) = (u_j, u_i) \text{ pro jisté } u_j \in U, \\ 0, & \text{v ostatních případech} \end{cases} \quad (4.10)$$

Definice 4.9: Nechť $G = \langle H, U, \sigma \rangle$ je orientovaný graf s množinou uzlů $U = \{u_1, u_2, \dots, u_n\}$. **Maticí sousednosti** grafu G nazýváme čtvercovou celočíselnou matici $V = [v_{ik}]$ řádu n , jejíž prvky jsou dány vztahem

$$v_{ik} = |\sigma^{-1}(u_i, u_k)|. \quad (4.11)$$

Matice V není symetrická

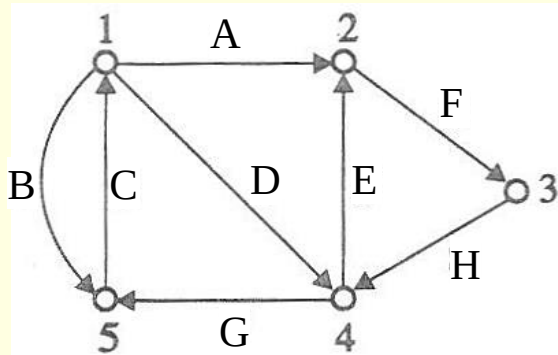
Matice dosažitelnosti $R = [r_{ik}]$:

$$r_{ik} = \begin{cases} 1 & \text{pokud existuje spojení z uzlu } u_i \text{ do } u_k, \\ 0 & \text{v opačném případě.} \end{cases}$$

Matice orientovaných grafů

incidence

sousednost



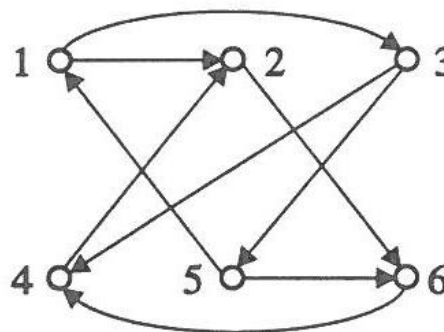
$A =$

	A	B	C	D	E	F	G	H
1	1	1	-1	1	0	0	0	0
2	-1	0	0	0	-1	1	0	0
3	0	0	0	0	0	-1	0	1
4	0	0	0	-1	1	0	1	-1
5	0	-1	1	0	0	0	-1	0

$V =$

	1	2	3	4	5
1	0	1	0	1	1
2	0	0	1	0	0
3	0	0	0	1	0
4	0	1	0	0	1
5	1	0	0	0	0

dosažitelnost



$R =$

	1	2	3	4	5	6
1	1	1	1	1	1	1
2	0	1	0	1	0	1
3	1	1	1	1	1	1
4	0	1	0	1	0	1
5	1	1	1	1	1	1
6	0	1	0	1	0	1

Spojová reprezentace grafů

Nevýhody maticové reprezentace grafu:
paměťové nároky a rozptýlenost uložené informace.

Základní forma **spojové reprezentace** neorientovaného grafu $G=(H, U, \sigma)$ sestává z pole Adj obsahujícího $|U|$ odkazů (ukazatelů) na seznamy sousedů jednotlivých uzlů z množiny U .

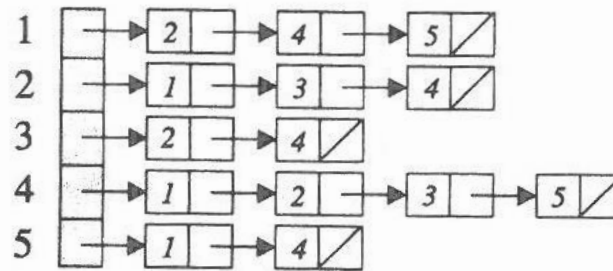
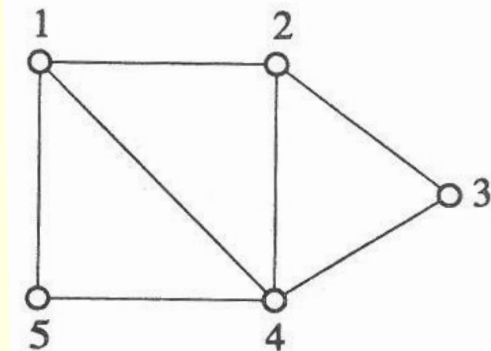
Pro každý uzel $u \in U$ obsahuje spojový seznam $Adj[u]$ jeden záznam pro každou neorientovanou hranu $h = [u, v]$ s druhým krajním uzlem v . Záznam typicky obsahuje identifikaci (číslo) uzlu v , případně identifikaci nebo doplňující údaje týkající se hrany h , pořadí záznamů je obecně libovolné.

Asymptotická paměťová složitost:

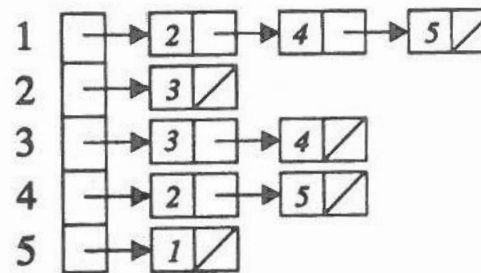
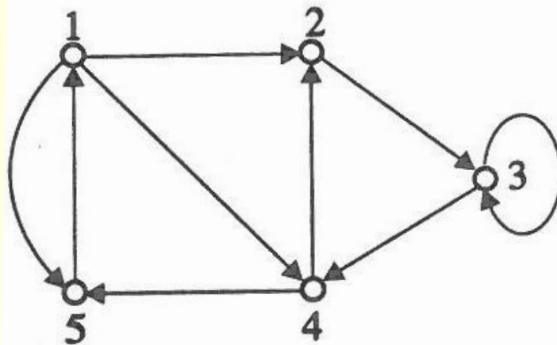
$$O(|U| + 2|H|) = O(|U| + |H|) = O(\max(|U|, |H|))$$

Spojová reprezentace grafů

(matice sousednosti:

$$V = \begin{array}{c|ccccc} & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 1 & 0 & 1 & 1 \\ 2 & 1 & 0 & 1 & 1 & 0 \\ 3 & 0 & 1 & 0 & 1 & 0 \\ 4 & 1 & 1 & 1 & 0 & 1 \\ 5 & 1 & 0 & 0 & 1 & 0 \end{array}$$


Pro orientovaný graf je uvažovat orientaci hran:



$$V = \begin{array}{c|ccccc} & 1 & 2 & 3 & 4 & 5 \\ \hline 1 & 0 & 1 & 0 & 1 & 1 \\ 2 & 0 & 0 & 1 & 0 & 0 \\ 3 & 0 & 0 & 1 & 1 & 0 \\ 4 & 0 & 1 & 0 & 0 & 1 \\ 5 & 1 & 0 & 0 & 0 & 0 \end{array}$$

Základní algoritmy

Prohledání grafu: systematická prohlídka všech jeho uzlů i hran. Prohledávací algoritmy jsou nejjednodušší a současně nejpotřebnější algoritmy na grafech.

Prohledávání do šířky

- angl. breadth-first search
- základní varianta prohledávání
 - kromě průchodu grafem je výsledkem strom hledání do šířky z daného uzlu (BF-strom), což je zároveň strom nejkratších cest z daného uzlu do všech dosažitelných uzlů
- vycházejí z něj další důležité grafové algoritmy
 - Dijkstrův algoritmus hledání nejkratší cesty
 - Jarníkův-Primův algoritmus hledání minimální kostry grafu
- budeme bez újmy na obecnosti předpokládat pouze obyčejné grafy (tj. bez smyček, jak neorientované tak i orientované)

Základní algoritmy

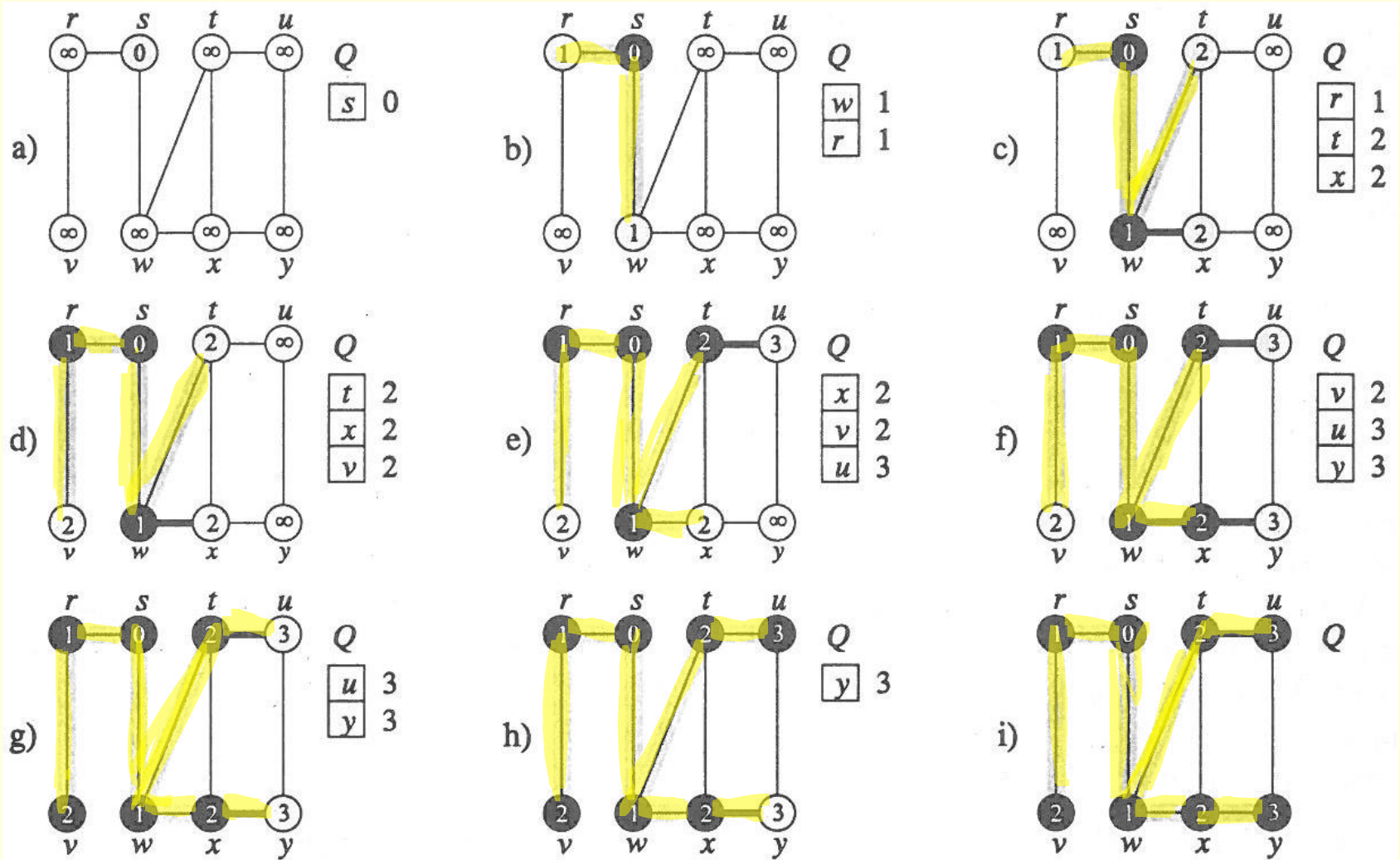
Prohledávání do šířky

```
BFS( $G, s$ )
1   for každý uzel  $u \in U - \{s\}$ 
2       do  $stav[u] := \text{FRESH}$ 
3            $d[u] := \infty$ 
4            $p[u] := \text{NIL}$ 
5    $stav[s] := \text{OPEN}$ 
6    $d[s] := 0$ 
7    $p[s] := \text{NIL}$ 
8   INIT\_QUEUE; ENQUEUE( $s$ )
9   while not EMPTY\_QUEUE
10      do  $u := \text{QUEUE\_FIRST}$ 
11          for každé  $v \in \text{Adj}[u]$  do
12              if  $stav[v] = \text{FRESH}$ 
13                  then  $stav[v] := \text{OPEN}$ 
14                       $d[v] := d[u] + 1$ 
15                       $p[v] := u$ 
16                      ENQUEUE( $v$ )
17          DEQUEUE
18           $stav[u] := \text{CLOSED}$ 
```

Všechny uzly mimo s označ jako nové, zatím nedosažitelné ze s a bez předchůdce. Samotný uzel s otevři s nulovou vzdáleností od s , rovněž bez předchůdce a ulož jej do fronty. Dokud fronta není prázdná, vyber z ní první otevřený uzel a jeho sousedy, kteří jsou noví, otevři, urči jim vzdálenost i předchůdce a ulož je do fronty. Odeber uzel u z fronty a zavři jej.

Základní algoritmy

Prohledávání do šířky



Základní algoritmy

Prohledávání do šířky

- pole vzdáleností $d[u]$ zachycuje vzdálenost každého uzlu z výchozího uzlu s

Věta 4.17: Nechť $G = \langle H, U \rangle$ je neorientovaný nebo orientovaný graf, na němž se provede prohledání pomocí algoritmu BFS z počátečního uzlu $s \in U$. Potom BFS nalezne všechny uzly $v \in U$ dosažitelné z uzlu s a po skončení bude $d[v] = d(s, v)$ pro každé $v \in U$. Kromě toho bude pro každý dosažitelný uzel $v \neq s$ nalezena nějaká nejkratší cesta z s do v taková, že ji tvoří nejkratší cesta z s do $p[v]$ prodloužená o hranu $(p[v], v)$.

- výsledný strom prohledání se nazývá též p-podstrom grafu G

Algoritmus 4.20 Určení cesty v p-stromu

CESTA(G, s, v)

1	if $v = s$	Cesta končí aniž začala
2	then write(s)	výstup počátečního uzlu
3	else if $p[v] = \text{NIL}$	v nemá předchůdce?
4	then write('cesta z ', s , ' do ', v , 'neexistuje')	
5	else CESTA($G, s, p[v]$)	Napřed cesta k předchůdci
6	write(v)	a pak koncový uzel v .

Základní algoritmy

Prohledávání do hloubky

- angl. depth-first search
- uzel má přiděleny dvě časové značky d – otevření, f – zavření
- výsledek je DF-les skládající se z několika DF-stromů
- pro neorientované grafy DF-les odpovídá komponentám grafu
- pro orientované grafy lze využít k rozkladu na silné komponenty: všechny uzly téže silné komponenty leží v témže p-podstromu

Základní algoritmy

Prohledávání do hloubky

DFS(G)

```
1   for každý uzel  $u \in U$ 
2       do  $stav[u] := \text{FRESH}$ 
3          $p[u] := \text{NIL}$ 
4    $i := 0$ 
5   for každý uzel  $u \in U$ 
6       do if  $stav[u] = \text{FRESH}$ 
7           then DFS-Projdi( $u$ )
```

Počáteční nastavení
všech uzlů na nové
a bez předchůdce.
Nastavení čítače značek
Zkus všechny uzly
a pro nové projdi
jejich potomky do hloubky.

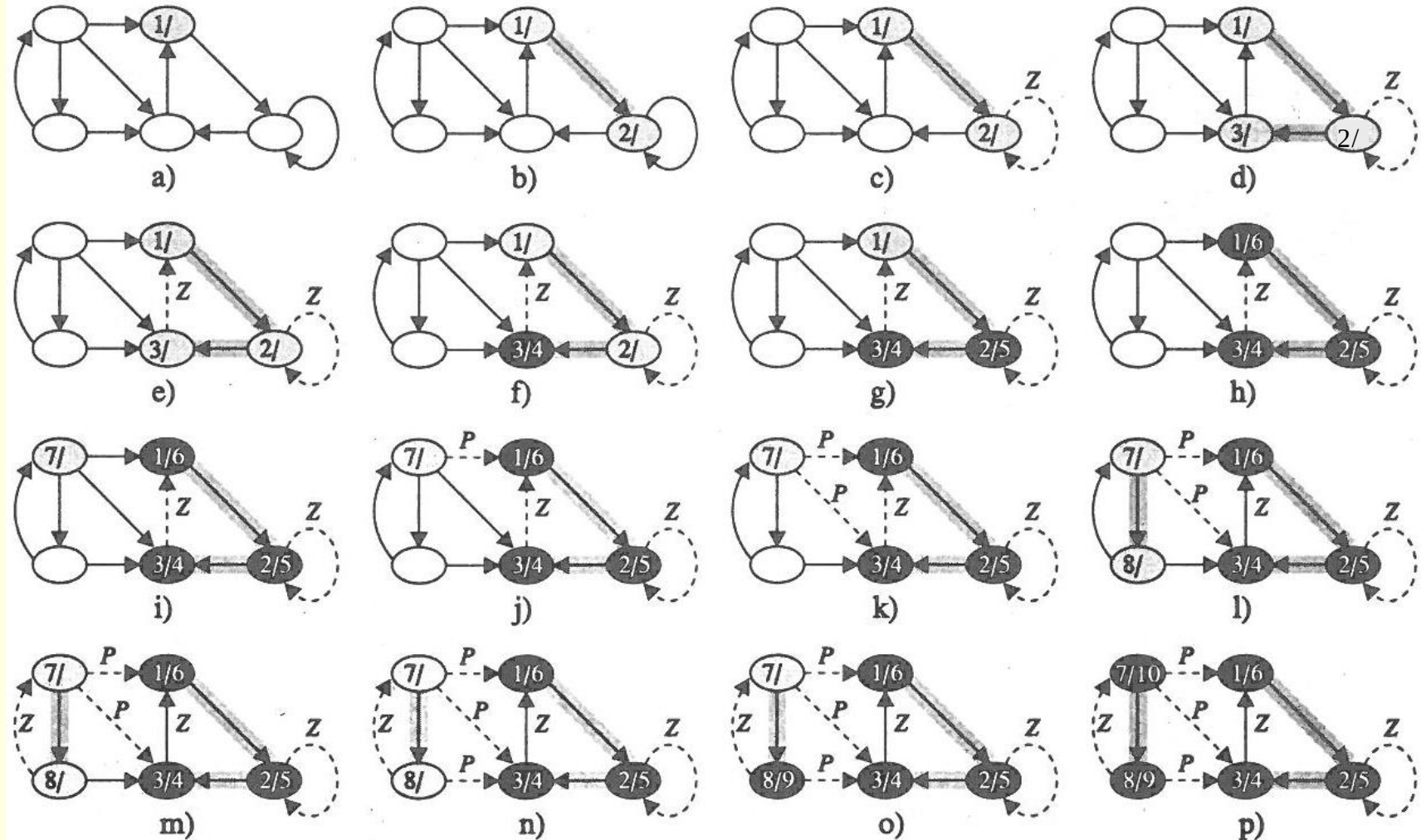
DFS-Projdi(u)

```
1    $stav[u] := \text{OPEN}$ 
2    $i := i + 1; d[u] := i$ 
3   for každý uzel  $v \in Adj[u]$ 
4       do if  $stav[v] = \text{FRESH}$ 
5           then  $p[v] := u$ 
6                 DFS-Projdi( $v$ )
7    $stav[u] := \text{CLOSED}$ 
8    $i := i + 1; f[u] := i$ 
```

Prohlas uzel u za otevřený
a přiděl mu první značku.
Projdi hranu (u, v)
a pro nový uzel v
stanov za jeho předchůdce u
a projdi potomky uzlu v .
Uzel u je dokončen,
dej mu koncovou značku.

Základní algoritmy

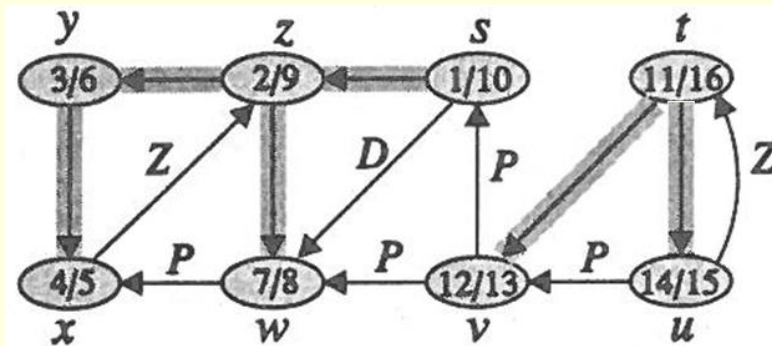
Prohledávání do hloubky



Základní algoritmy

Prohledávání do hloubky

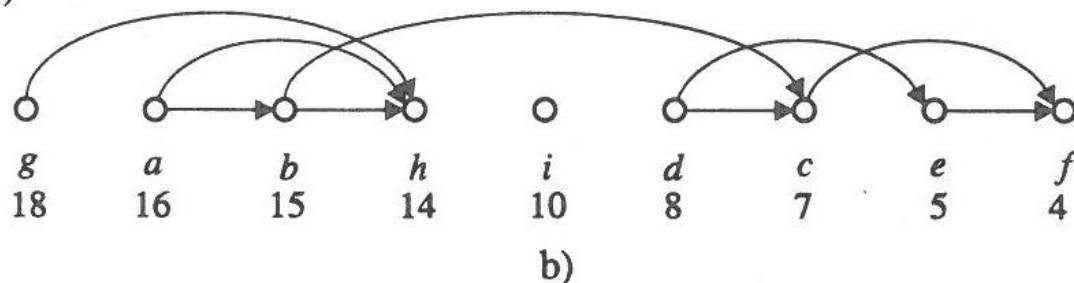
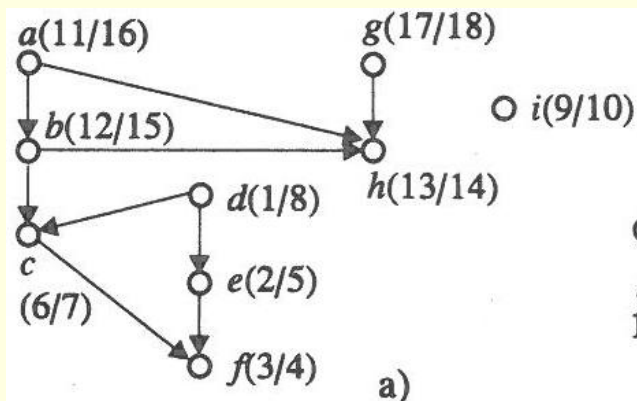
- algoritmem lze klasifikovat hrany grafu (viz symboly v obr):
 - ◇ **Stromové hrany** jsou hrany tvořící DF-leš G_p . Hrana (u, v) je stromovou hranou, jestliže k objevení (otevření) uzlu v došlo při procházení seznamu sousedů uzlu u .
 - ◇ **Zpětné hrany** jsou takové nestromové hrany (u, v) , které vedou od uzlu u k nějakému jeho předkovi v DF-lese. Případné smyčky považujeme rovněž za zpětné hrany.
 - ◇ **Dopředné hrany** jsou takové nestromové hrany (u, v) , které vedou od uzlu u k nějakému jeho potomkovi v DF-lese.
 - ◇ **Příčné hrany** jsou všechny ostatní hrany. Mohou spojoovat dva uzly téhož DF-stromu (pokud jejich počáteční uzel není předkem koncového) nebo uzly ležící v různých DF-stromech.



Základní algoritmy

Topologické uspořádání

- úplné uspořádání uzlů, v němž je pro každou hranu (u, v) uzel u před uzlem v
- lze zavést pouze pro acyklické grafy
- odpovídá sestupnému seřazení okamžiků uzavření při DFS
- není obecně určeno jednoznačně (záleží na tom, kterým uzlem začneme)



Základní algoritmy

Testování cyklů v grafu

- Orientovaný graf G je acyklický právě tehdy, když se při jeho prohledání do hloubky neobjeví žádná zpětná hrana.
- Pokud orientovaný graf nemá žádný list (výstupní stupně všech uzlů > 0), pak obsahuje cyklus.

Základní algoritmy

Silné komponenty

- mnoho grafových úloh se řeší rozkladem na podproblémy odpovídající jednotlivým silným komponentám s následnou kombinací výsledků těchto podproblémů
- takový postup naznačilo i zavedení pojmu kondenzace orientovaného grafu

Algoritmus 4.31 Rozklad na silné komponenty

S-COMP(G)

- 1 proved' DFS(G), který určí časovou značku uzavření $f[u]$ každého uzlu u
- 2 vytvoř opačně orientovaný graf G^-
- 3 proved' DFS(G^-), přitom v hlavním cyklu DFS probírej uzly v pořadí klesající hodnoty $f[u]$ (získané v kroku 1)
- 4 rozklad množiny uzlů podle jednotlivých stromů DF-lesa získaného v kroku 3 určuje rozklad na silné komponenty